Using Jev with Claude

A practical handbook for setup and everyday workflows

Use Claude to design and maintain the workflow, Jev to make bounded semantic judgments, and ordinary code to calculate, enforce policy and execute actions. Start with one read-only batch task. Add hooks, model routing or a dashboard only after that task has proved useful.

This companion to the Jev Introduction and Knowledge Appendix assumes basic familiarity with Jev. It focuses on making the combination work: choosing an integration, defining questions, preparing data, checking results and operating it reliably. The examples are general educational exercises, not integrations installed in your environment.

Read for	Pages
Choose an integration and complete setup	2-3
Design questions and build a support triage pilot	4-6
Apply the pattern to bulk work, code and retrieval	7-9
Use prompt hooks and the optional model router	10-11
Explore browser use, voice and a custom dashboard	12-13
Evaluate, operate and troubleshoot	14-16
Decide whether to add GPT-6 Astra	17
Sources, documentation links and reusable knowledge	18-23

How to read the evidence

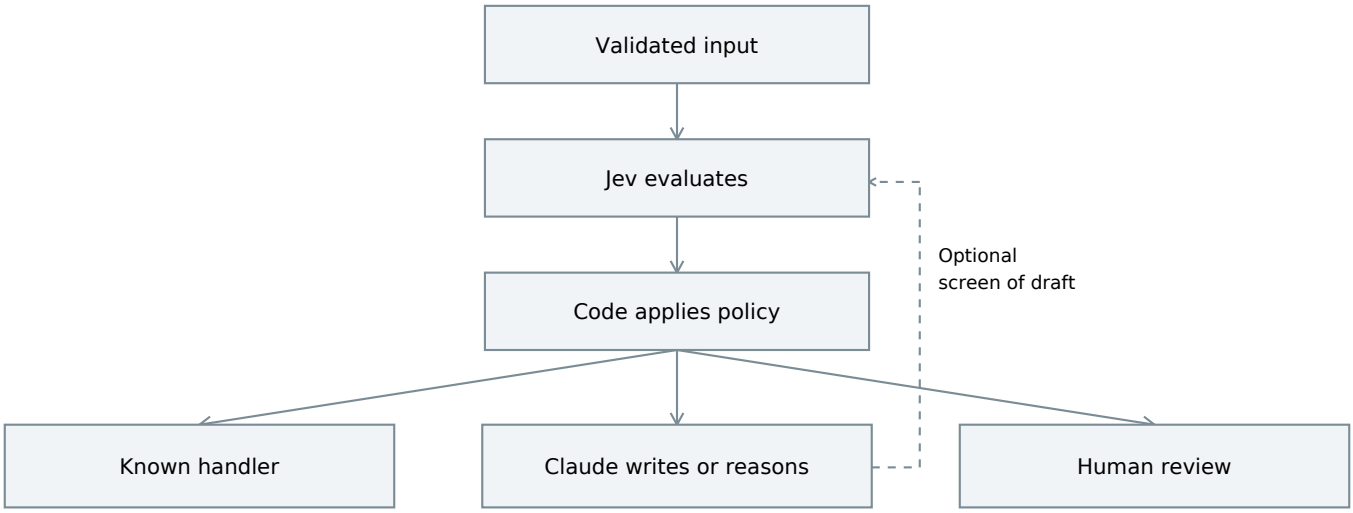
PARAPHRASE identifies what the supplied sources report. **EXTERNAL VERIFICATION** means a current primary source documents the point; it does not mean an independent performance test. **INTERPRETATION** marks analysis. **TEACHING CHOICE** marks the handbook's exercises, policies and example thresholds. **MISSING CONTEXT** flags unresolved facts. No direct source quotations are needed.

H01-H12 identify segments of the attached transcript collection. D01-D24 identify linked references. These IDs are local to this handbook and do not imply a crosswalk to the earlier introduction. References were checked on **1 October 2026**. All sample probabilities and test policies are illustrative unless explicitly attributed. No live Jev, Claude or Astra API trials were run for this handbook.

Choose the integration you need

EXTERNAL VERIFICATION D01-D03 and D20 | INTERPRETATION of H04-H12

Your aim	Integration	What it changes
Have Claude build a Jev workflow	Official TypeSafe skill in Claude Code	Teaches the coding agent how to use the API; does not route every prompt.
Classify or score records in your software	Application calls the Jev API	A program submits state and questions, then interprets the results.
Answer known read-only queries before Claude runs	Custom prompt hook or app dispatcher	Selects an approved function; code retrieves data and formats the answer.
Choose Claude model or reasoning effort	Third-party jev-model-router mod	Changes selected model-request parameters. Separate from the official skill.
Use Jev from ordinary Claude chat	An environment with tools and API access	A pasted skill alone does not supply execution, network access or credentials.



Solid arrows: documented division of responsibilities, adapted to the handbook. Dashed arrow: optional draft-screening design. The diagram shows an application pattern, not the internal architecture of Jev. [D01, D08, D15; TEACHING CHOICE]

Recommended first route. Use Claude Code with the official skill, then build a small CSV-to-CSV classifier. It is easy to inspect, has no automatic external actions, and reveals whether Jev helps your task. Ordinary Claude chat can help write the requirements even when it cannot execute the integration.

Do not conflate the products. Claude is a model family; Claude Code is an agent environment; the TypeSafe skill is guidance; Jev is the decision service. A dashboard is an application around these parts. The source collection sometimes calls this an “agentic OS”; that does not mean a new operating system. [H08, H10]

Set up a first working project

TEACHING CHOICE for the setup checkpoints | EXTERNAL VERIFICATION D02-D05

The official pages already document installation and SDK setup in detail. Use the linked instructions below rather than a copied command sequence that can become stale. Complete the checks in order; stop at the first unmet prerequisite.

Step	Follow the existing documentation	Evidence you should see
1 Prepare	Use an installed, signed-in Claude Code environment and a disposable project folder. Keep source control or a backup.	Claude can inspect and edit this project.
2 Enable guidance	TypeSafe agent skill [D02]. Choose one installation method.	The skill is available; invoke `/typesafe:typesafe-ai` if needed.
3 Obtain access	TypeSafe Quick start [D04]. Create an account and API key through the console.	An authorized key exists; do not assume a video's credit or waitlist offer still applies.
4 Make credentials available	Use the documented TYPESAFE_API_KEY environment variable through your local secret handling.	The child process can detect that the variable is set without displaying its value.
5 Run one query	Python or JavaScript SDK guide [D05]. Let Claude implement one documented example with synthetic input.	Raw typed answer, actual model version, elapsed time and reported usage.
6 Save the baseline	Record SDK version, question text and the result; separate secrets from project files.	A repeatable smoke test and a known way to disable API calls.

Prompt to adapt

Use the TypeSafe skill and current official documentation. In this disposable project, create one smoke test using a synthetic ticket: “I was charged twice for the same subscription.” Ask one Noul question about whether it reports a billing problem. Use the API key already supplied through the environment; never print it. Show the documented request fields, raw answer, resolved model version and usage. Make no external changes. If documentation or access is unavailable, report the exact blocker and do not invent an API contract.

Before the first batch

Keep test inputs synthetic or approved for the provider. A local script still sends its request to the configured API. A .env file needs a loader and must be excluded from source control; its mere presence does not initialize the client. These are project setup responsibilities, not functions supplied by a skill.

Turn a task into a decision contract

TEACHING CHOICE adapted from H04 H06 H10 | EXTERNAL VERIFICATION D06-D09

Start with the action you want your software to take. Work backwards to the evidence and the smallest judgment it needs. “Analyze all our customers” is not a usable contract; “identify messages that explicitly request a refund” is.

Field	Write this before asking Claude to implement
Unit and state	One ticket, comment, passage or function at a time, with stable ID and the relevant context. Include policy excerpts only when needed.
Question	One specific judgment. Give positive examples, exclusions and a genuinely ambiguous example.
Allowed result	Choose Noul, Choice or Score. For Choice, include an unknown or other category when the real data can fall outside the list.
Policy	Define the code branch, the review path and what must never happen automatically.
Evidence and version	Keep source IDs, question version, raw outputs, model version and human correction separate.
Success	Name the errors that matter, acceptable review workload, end-to-end time and cost budget.

Primitive	Use when	Reading the answer
Noul	A statement can be evaluated as yes or no.	The value estimates P(yes). A value near zero suggests no; it is not low confidence in the answer.
Choice	One label or handler should be selected.	Read the selected option, option probabilities and separate confidence field.
Score	Levels describe an ordered rubric.	A score places the item on that rubric. It is not automatically a probability of an event.

TypeSafe documents Choice and Score confidence as a statistic derived from their probability distributions. Do not assume it equals the largest option probability, or that a confidence value of 0.9 means 90% empirical correctness. Test those relationships on your own task. [D07]

Keep dependent work in separate stages

Ask independent questions about the same record together. A question in the same request cannot read another question's answer. If Jev selects a document that you must then retrieve, retrieve it before a second evaluation. If Claude drafts a reply, the draft must exist before Jev can screen it. Parallel evaluation also does not imply statistically independent errors. [D06, D08; INTERPRETATION]

Prompt to adapt

Before coding, write the decision contract: one record schema, atomic questions, allowed labels or rubric levels, exclusions, missing-data behavior, policy branches and measurable acceptance criteria. Keep exact calculations in code. Highlight any question that needs evidence we do not have.

Build a support triage pilot

TEACHING CHOICE | Inspired by H05 H06 and the documented patterns D08-D09

The first exercise sorts synthetic support tickets and prepares a review queue. It produces files for inspection; it does not send replies, issue refunds or change customer accounts. This combines a bounded classification task with a clearly separated Claude drafting task.

Prepare the input

Use CSV columns `ticket_id, subject, message`. Give every row a unique ID. Keep the original file unchanged. Claude should validate empty messages and duplicate IDs before making calls. These five fixtures are a functional smoke test, not an accuracy benchmark.

ID and message	Expected handling
T01 “Charged twice. Please refund the second payment.”	Billing; refund signal; mandatory human review.
T02 “I cannot sign in and cannot use the service.”	Access; substantial impact; draft assistance only if approved guidance exists.
T03 “Please add dark mode.”	Feature request; feature log.
T04 “Great session yesterday, thanks!”	Other; no invented support problem.
T05 “Cancel my plan. Also, I cannot log in.”	Mixed intent; review rather than forcing one confident route.

Define three questions

Category as Choice. Use billing, access, feature, other and mixed_or_unclear. Billing covers charges and payments. Access covers authentication and entry to the product. A substantial combination belongs in mixed_or_unclear. Classify the stated problem; do not infer a purchase history.

Refund request as Noul. True means explicitly asking for money back or a credit on money already paid. A cancellation request alone is not a refund request. Keep this independent of the category so a misclassified label cannot hide a refund signal.

Product impact as Score. Define 0 as no product impairment stated, 1 as inconvenience with a workaround, 2 as a core task blocked, and 3 as the service unusable or a serious continuing loss described. Treat anger separately if you later need it; emotion is not the same as impact.

Prompt to adapt

Implement a dry-run support triage pilot from this contract. Preserve all ticket IDs. Ask the three questions in one request per valid ticket, using bounded concurrency. Save raw answers and a separate policy result to JSONL; export triage.csv and review.csv. Every input must end as completed, review or explicit error. Use synthetic fixtures first. Do not draft messages until the triage output has been inspected.

Checkpoint. Confirm the refund reaches review, the mixed request is not silently reduced to one intent, and every row is accounted for. Never invent expected numeric scores for a live run; compare the actual outputs and policy outcomes.

Apply policy and add Claude drafting

TEACHING CHOICE | Illustrative thresholds, not validated deployment settings

Keep the decision policy in a small, versioned function outside the model prompt. The following policy prioritizes uncertainty and financial requests over normal routing. “Review” is a destination for the application, not a new Jev primitive.

```
if missing_input or api_error or invalid_response:
  queue_for_review(reason="input_or_service_error")
elif category == "mixed_or_unclear" or category_confidence < 0.80:
  queue_for_review(reason="unclear_route")
elif refund_noul >= 0.20:
  queue_for_review(reason="possible_refund")
elif impact_confidence < 0.80 or impact_score >= 2.0:
  queue_for_review(reason="uncertain_or_serious_impact")
elif category == "feature":
  append_to_feature_log(ticket_id)
else:
  create_draft_task(ticket_id)
```

The low refund threshold deliberately sends uncertain cases to review. These values only illustrate branching; tune them on labelled examples before using real records. Equality belongs to the branch shown. Missing confidence is an error or review case, not an implied high-confidence result.

Give Claude the material needed to write

A draft task should contain the original ticket, approved support guidance, the route and relevant evidence. Do not replace the ticket with a label alone. Claude writes a proposed response with no unsupported commitments. Jev can then screen narrow properties, such as whether the draft promises a refund or fails to address an explicit request. That screen cannot prove correctness. [H06; D15]

Prompt to adapt

For draft-eligible tickets, prepare replies using only the approved support guidance. Cite the guidance ID in a separate review field. Save drafts; do not send them. Then use Jev to screen each original-ticket and draft pair for an unsupported financial promise and an unanswered explicit request. Keep the raw screen results. Route suspicious, missing-evidence or uncertain cases to a person. A passed screen must not count as permission to send.

Inspect the whole result

Check	Pass condition
Completeness	Input count = completed + review + explicit errors; no dropped or duplicated IDs.
Policy trace	Every route has a machine-readable rule ID and the values that triggered it.
Draft grounding	Each substantive answer can be traced to provided guidance; no invented account action.
Failure rehearsal	A forced timeout or missing field produces review, not an empty success record.

Save human corrections with reasons. Later use them to improve questions and thresholds on a development set; keep a separate test set untouched. Changing question text is workflow configuration, not retraining Jev's weights. [D10; INTERPRETATION]

Apply Jev to bulk business work

PARAPHRASE H02 H05 H07 | TEACHING CHOICE for the adaptations below

The transcripts repeatedly show large batches of comments, leads and support records. Their useful contribution is the division of labour: Jev narrows the pile, code aggregates it, and Claude writes or investigates selected records. Reported timings are demonstrations, not your capacity plan.

Workflow	State and decisions	Useful output and boundary
Comment triage	Comment plus relevant parent thread. Separate spam, question, constructive feedback and buying intent.	An ID-based shortlist for replies. Several properties can coexist; do not force positive feedback and an unanswered question into one exclusive label.
Lead qualification	Supplied company and role facts; Noul for each eligibility signal and Score for documented fit.	Ranked review list. Compute explicit country and numeric filters in code. Missing role or budget evidence remains unknown.
Community or call analysis	One post or a relevant transcript segment with topic context. Choice for topic; separate urgency.	Topic dashboard computed from labels. Report reviewed and uncertain counts alongside totals.
Churn prioritization	Verified activity features plus a defined qualitative rubric.	A contact-priority score. Do not label it “probability of churn” without outcome-based calibration.
Invoice screening	Parsed invoice fields, trusted supplier reference and relevant text.	Suspicion flags for review. Code checks totals and exact bank-detail differences; humans authorize financial actions.

Run a representative pilot

Select a small batch that includes short and long records, missing fields, multiple languages and ambiguous cases. Label it before looking at Jev's results. Ask Claude to preserve raw source text and produce a comparison view showing disagreements and omissions. Expand only after understanding those failures.

Prompt to adapt

Use the existing Jev integration to triage this approved dataset. First identify fields that code can evaluate exactly and fields needing semantic judgment. Propose separate questions for properties that can coexist. Process a representative sample, retain uncertainty and errors, and produce an ID-based review file. Calculate totals in code. Estimate full-run cost from measured input usage, and keep Claude writing costs separate.

When packaging the workflow as a skill

Store the task contract, input schema, scripts, question version, evaluation fixtures and output specification together. A skill describes how to run the process; it is not the scheduler or the API credential store. H05's recurring churn example needs a separately configured scheduler and a process that can access both the data and the key.

Use Jev around software development

PARAPHRASE H04 | TEACHING CHOICE for a controlled implementation

A useful first coding workflow is comment review: Jev flags a shortlist, Claude examines those candidates and proposes edits. Start with advisories. A semantic screen does not replace a compiler, tests, static analysis or a security review.

Recipe for comment cleanup

1. Extract comments with their enclosing function, nearby code and stable file location. Exclude generated files and vendored dependencies. A comment without its code is insufficient evidence for accuracy.

2. Ask separate questions: does the comment contradict the provided code; does it add information beyond restating the code; does it document a constraint or reason that should be preserved? Preserve uncertainty. “Obvious” is a rubric decision, not proof that deletion is safe.

3. Send the shortlist and surrounding code to Claude. Request proposed diffs, not blanket deletion. Re-run relevant checks and inspect whether behavioural explanations were lost. Review a sample of the unflagged comments to estimate misses.

Candidate screen	Better boundary
Function naming	Ask whether the name reflects the supplied behaviour and side effects. Wider behaviour may require call-site context.
Logging content	Flag likely sensitive values for investigation. Keep deterministic secret scanning and access controls.
Code smells	Use parsers or static analysis for countable structure; use Jev for narrow semantic concerns with enough context.
Pull request triage	Score defined risk dimensions per change; Claude investigates. Preserve cross-file dependencies and unflagged sampling.
Test quality	Screen for suspicious weakening, then compare tests and execute them. A model's approval is not a passing test.

Prompt to adapt

Use the TypeSafe skill to design an advisory comment review for this repository. First list what static tools can establish. Extract a small sample with enclosing code, define separate accuracy and usefulness questions, and show the rubric for review. Return evidence-linked candidates. Ask Claude to propose edits only to selected comments; preserve rationale and invariants. Report missed issues from a sample of unflagged comments as well as false alarms.

Refine without creating a self-confirming loop

Use Claude to explain disagreements and propose better questions. Approve a new rubric version, then evaluate it on held-out cases. Do not let the same model generate all labels, tune the policy and declare the result correct. “System 2 trains System 1” in H04 usually describes changing criteria or orchestration; it does not establish weight training.

H04 also proposes large browser-testing swarms and broad security screening. Those are exploratory extensions. Neither the transcript's projections nor agreement with another model establishes defect recall or production readiness.

Improve retrieval and evidence handling

EXTERNAL VERIFICATION D12-D15 | TEACHING CHOICE for adoption

Detailed implementations already exist for these tasks. Follow the linked cookbooks and adapt their inputs and evaluation sets; the handbook adds selection criteria and checks rather than repeating their scripts.

Task and detailed recipe	Use it for	Check before adoption
Skill suggestion [D12]	Choosing a relevant skill from a roster.	The published Hermes example retains the full roster and adds a suggestion. It does not prove that deleting Claude's skill index saves a fixed number of tokens.
Classifying RAG passages [D13]	Screening retrieved passages before Claude answers.	Measure recall of necessary evidence, not just how much context was removed. Keep source IDs and an uncertainty path.
Double checking citations [D14]	Comparing a claim or quotation to an available source.	The source must be retrieved. A citation screen cannot establish the source's own truth or discover every missing qualification.
Guardrails for LLMs [D15]	Adding narrow input or output screening.	Treat it as one fallible detector. Keep authorization and executable-action restrictions outside the model.

Recipe for a passage filtering pilot

Run ordinary retrieval first and retain its candidate IDs. Give Jev the user question and each candidate passage with enough neighbouring context to judge relevance. Keep high-relevance passages and uncertain ones for the first pilot. Let Claude answer from the retained evidence and record citations. Compare with an unfiltered baseline on the same questions.

Inspect cases where the answer worsened because an exception, definition or contradictory passage was removed. Set the filter policy from the cost of missing that evidence. A smaller context is useful only when the final answer remains supported.

Prompt to adapt

Adapt the official passage-classification cookbook to a read-only pilot. Keep original retrieval results, passage IDs and relevance outputs. Compare Claude answers with and without filtering against a labelled question set. Report evidence recall, answer correctness, input tokens and total latency. Include an option that preserves uncertain passages. Do not treat agreement between Jev and Claude as ground truth.

A disagreement to preserve

H07 says Jev must never judge another model's answer; H04 and H06 use verification screens, and the citation cookbook supplies a bounded example. The useful distinction is between checking one property against supplied evidence and certifying an entire answer. The latter remains unsupported. [INTERPRETATION]

Build a read only fast path

PARAPHRASE H09 | EXTERNAL VERIFICATION D16 D21 | TEACHING CHOICE

The “Jev answers database questions” demo can be understood as selecting an existing query and bounded arguments, executing it in ordinary code, then formatting its result. Jev does not gain unrestricted database knowledge when you install a skill. Nor should it compute a percentage from memory.

Start with a dispatcher before adding a hook

Define a small registry such as `count_comments`, `follower_change` and `unsupported`. Each handler has an input schema, approved data source and deterministic formatter. Choose dates from a known set only if the application validates their meaning and timezone. Other requests go to Claude or clarification.

Stage	Responsibility
Jev	Select the supported intent and bounded arguments from supplied options; return uncertainty.
Code	Check permissions, required arguments and valid combinations; execute read-only parameterized queries; calculate and format results.
Claude	Handle requests outside the registry, ambiguous requirements or explanations that need synthesis.
Operator	Verify provenance, denominator, date window and a way to disable the fast path.

For a survey question, code filters the relevant population and counts the matching records. Display numerator, denominator, treatment of missing responses and the computed percentage. The percentage shown in H09 is a demo result; it is not evidence that Jev itself performed the arithmetic.

Understand the hook boundary

Claude Code documents `UserPromptSubmit` as running before model processing. It can add context or block the prompt. A normal successful hook output can still be fed to Claude; it does not by itself prove that a model request was avoided. A custom short-circuit path must explicitly implement and test display, control flow and transcript behaviour. A coloured answer is not evidence of zero token use. [D21]

Prompt to adapt

Build a read-only dispatcher for three explicitly listed queries and an unsupported outcome. Use Jev only for intent and bounded argument selection. Keep SQL, arithmetic, permission checks and formatting in code. Test invalid argument combinations, an unknown query, empty data and an API timeout. First expose it as a command. Then assess whether the installed Claude Code hook API can intercept and display the answer without a Claude call; prove that with a request trace.

MISSING CONTEXT. H09 does not provide its full hook implementation or a verified downloadable prompt URL. Do not assume its “Jev first” prompt is portable to Cursor, Codex or Claude Desktop. Link to the official hook reference for the environment actually used.

Try the optional Claude model router

PARAPHRASE H11-H12 | EXTERNAL VERIFICATION D20 | Optional third-party integration

The supplied collection already contains extensive documentation for `jev-model-router`. Read its installation, options and diagnostics page [D20] rather than duplicating that reference here. It is distinct from the official TypeSafe skill and from the read-only query dispatcher.

What to know before following that guide

The maintainer marks the mod early access and lists Claude Code 2.1.259 or newer plus the function-hooks feature flag. Verify that your installed version supports the documented interface. Those are version-specific maintainer requirements, not a guarantee for every Claude installation. [H12, D20]

Behaviour in the supplied documentation	Practical implication
Subagent model routing on; main effort routing on; main model routing off by default.	A changed effort is not evidence that the main conversation changed model.
TypeSafe, Gateway and built-in classifier paths differ.	A functioning router may be using the built-in classifier rather than Jev. Inspect the backend log.
Main-model changes may invalidate prompt caching.	Compare total session cost. A cheaper per-token model can be a more expensive session.
API failures leave the original request unchanged.	This is an optimization fallback. It is not a security gate that blocks risky work.
Configuration keys depend on how the plugin is loaded.	Follow the matching install path and options section; a wrong key can silently retain defaults.

Run a comparison that can justify keeping it

Save a baseline run on a representative task set: small edits, ambiguous requests, multi-file changes and tasks with external effects. Repeat with the router enabled, preserving the same evaluation criteria. Inspect the logged proposed tier and actual applied change. Check task correctness, rework, latency and total token charges; routing down is not itself a success metric.

Test a missing key and a forced service failure. Confirm whether the backend changed or the original request ran unchanged. Restore the baseline configuration when the experiment is over unless the comparison supports keeping the mod. Do not infer effective request settings from an unchanged session header alone. [H12]

Do not mix confidence scales

H12 describes a native TypeSafe confidence field, an optional Gateway distribution and a built-in path without confidence. The router's derived Gateway value is not automatically equivalent to TypeSafe's confidence statistic. Re-evaluate policy after changing backend. Also interpret “nothing leaves the machine” narrowly: a fallback inside an agent may still use the agent's hosted model.

Explore browser voice and media workflows

PARAPHRASE H02-H06 H08 | EXTERNAL VERIFICATION D17 | TEACHING CHOICE

Browser actions

The Jev Ultrafast repository [D17] provides setup and an inspector. Its documented loop turns visible controls into an indexed action space; Jev selects an operation and target, while a small generative model supplies text when needed. Its flight example stops at visible search results, not a purchased ticket.

Follow that repository's current setup. For a first exercise, use a local test page with a few links and a search field. Set a read-only goal, a maximum step count and a stop condition. Inspect the observed controls, selected target, actual action and final page. A valid action ID can still be the wrong action.

Prompt to adapt

Adapt the documented browser example to a disposable local page. Let Jev select only observed controls. Stop on a stale target, repeated failure, unclear action or the step budget. Keep text generation separate and show its cost. Verify the final page state independently. Do not use a logged-in production account, submit purchases or send messages in this exercise.

Voice is a separate input and output layer

H02 uses a speech service; H08 describes a local voice setup. In either case, speech recognition produces text, Jev can route that text, a handler or Claude performs the task, and speech synthesis reads a response. Jev itself is not the speech recognizer or voice generator. Include transcription uncertainty and incomplete utterances in tests.

Image search and creative asset selection

H02 and H07 describe searching descriptions or metadata about images. Give each asset an ID and a verified description; Jev can score the descriptions against a request, then code selects a shortlist. Keep the original image available for final inspection. Text relevance does not establish visual quality, accessibility or brand fit.

Transcript attraction	Usable scope
Jev “edits” a reel	It can select among supplied assets or editing choices. Other software cuts, renders and exports.
Instant voice actions	A bounded application may route recognized requests. Confirm irreversible actions separately.
Game strategy plus fast actions	A sandboxed planner-controller experiment, not evidence for real traffic or autonomous driving.

Measure the complete chain: transcription or observation, network calls, decision, execution and rendering. Fast model inference alone does not establish fast application response. [INTERPRETATION]

Package a workflow into a useful dashboard

PARAPHRASE H08 | TEACHING CHOICE for architecture and operation

H08 presents a visual layer, a file-based memory layer and a skills layer. The practical value is making repeatable work visible and accessible. A dashboard does not make the underlying tasks reliable, and an Obsidian graph view is not automatically a retrieval system.

Component	Minimum useful responsibility	Keep distinct
Dashboard	Start a named job; show status, review items and links to results.	Do not embed API secrets in client-side code.
Workflow service	Validate inputs, run steps, store status and enforce allowed actions.	A button is a request to run; it does not grant extra permissions.
Skills and scripts	Describe and implement each reusable task with declared inputs and outputs.	A skill is not a persistent background process.
Memory and records	Store source documents, IDs, timestamps and run outputs for retrieval.	File links, search indexes and a semantic graph are different mechanisms.
Scheduler	Launch approved jobs on a schedule with retries and a run identity.	Scheduled availability depends on where the process runs.

Build the interface after the job works

First run the workflow from a command and confirm its outputs. Then add a button that invokes the same implementation. Display queued, running, review_required, completed and failed states. Link each result to its input records and execution log. Do not show “completed” merely because a model returned successfully.

Make recurring runs idempotent

An idempotent job can be retried without duplicating its effects. Use a stable record ID and a key such as workflow version plus data interval. Reuse completed classifications when policy allows; retain failed IDs for retry. Prevent overlapping scheduled runs from duplicating drafts or spreadsheet rows. Record both scheduled time and the data interval actually processed.

Prompt to adapt

Package the proven triage script as a reusable workflow with a documented input schema, output schema, policy version and dry-run mode. Add a minimal local dashboard that calls the same script, shows per-record errors and links to review.csv. Then describe the scheduler configuration required for recurring runs, including credentials, overlapping-run protection and retry behaviour. Do not claim the schedule exists until it is configured and verified.

Sharing with a team

Give each person only the workflows and data they need. Keep provider credentials on the service side. Establish who owns the review queue and who can approve actions. The transcript's paid templates and community offers are optional products; they are not prerequisites for the architecture described here.

Evaluate quality before measuring savings

TEACHING CHOICE | Motivated by H04 H07 H10 and documented limitations D11

Separate a functional test from a quality evaluation. Five fixtures can show that the program runs; they cannot establish accuracy. Use representative labelled records, including edge cases and rare costly errors. Separate development cases used for tuning from a held-out test set.

Measure	Why it matters
Per-label precision and recall	Precision counts correct flags among flags; recall counts detected relevant cases among all relevant cases. High accuracy can hide missed rare issues.
Automatic coverage and review load	Measure what fraction receives an automatic decision and how many records a person must inspect.
False automatic actions	Count wrong decisions that crossed the action threshold, separately from safe referrals to review.
End-to-end completion	Require every record to finish or have an explicit error; include timeouts, retries and omissions.
Latency and full cost	Record median and tail latency, total wall-clock time, provider usage and downstream Claude work.
Stability and drift	Repeat boundary cases and compare new model, question and data versions against the saved baseline.

Test probability and confidence separately

For a Noul such as “explicit refund request”, group predictions into probability bins and compare the average prediction with the observed yes frequency. Keep enough cases per bin and report sample size. For Choice, test label quality and the empirical error rate of each confidence band. A concentration statistic is not automatically a probability of correctness. For Score, inspect rubric agreement and ranking quality; do not interpret 2.8 as 28% risk. [D07; TEACHING CHOICE]

Design a fair comparison

Compare Jev plus code plus Claude with the current workflow on identical inputs and completion criteria. Include deterministic rules as a baseline when applicable. If one path skips records or uses less evidence, its lower cost is not a like-for-like improvement. Other-model agreement is useful for locating disagreements, not establishing gold labels.

Prompt to adapt

Evaluate this frozen workflow on the held-out labelled records. Report per-class confusion counts, precision, recall, automatic coverage, review rate and errors among automatic decisions. Include all failed records. Compare total latency and cost against the existing process. Show probability calibration and confidence-band error rates separately. Identify the highest-impact failures before recommending any threshold change.

Choose the release condition before seeing the final result: for example, no loss of required records, an acceptable recall for costly cases and a review workload the team can handle. A finite test never proves zero future errors.

Estimate costs and operate within limits

EXTERNAL VERIFICATION D10 | TEACHING CHOICE for estimates and controls

On 1 October 2026, TypeSafe's model page lists Jev 1.13.0 at **$0.042 per million input tokens**, with output tokens free. It lists 64k total request tokens and 32k for state plus the longest question. Published rate limits are 40 requests and 100k tokens per second and are explicitly subject to change. Recheck the linked Models page before sizing a run. [D10]

An input-only estimate is not a project budget

```
Jev cost = measured billable input tokens / 1,000,000 × current rate
Total = Jev + Claude + other model calls + tools and infrastructure
```

Illustration: 10,000 records averaging 1,500 billable input tokens use 15 million tokens. At the listed rate, the Jev component is $0.63 before retries. This is arithmetic under an assumed workload, not a measured result. Use API-reported usage to replace the assumption. Question text also contributes to the input budget.

Control concurrency instead of launching everything

Parallel questions within one request and concurrent requests across records are different. Use a bounded worker pool and queue. Account for both token and request limits, honor retry guidance and keep a total retry budget. Under sustained errors, stop new work and retain the remaining record IDs. Measure useful completions per second rather than requests launched.

Operation rule	Reason
Record actual model version	Aliases can move. Thresholds validated on one version may change behaviour on another.
Bound each request and the whole run	A fast average does not prevent a stalled request or an unexpectedly expensive retry loop.
Write progress incrementally	A restart can resume failed or pending records without repeating completed work.
Store raw result plus policy result	A wrong model answer and a wrong application branch need different fixes.
Separate optimization from permission gates	A router outage may retain Claude; an unavailable authorization check must not authorize an action.

The transcript claim that extra questions are “free” is too broad. Shared state and parallel evaluation can reduce repeated work; question tokens and service limits still matter. Likewise, “no Claude call” does not mean zero Jev cost. [H06, H09; D08; INTERPRETATION]

Correct the support demo arithmetic

H06 gives 50 emails, “0.026” cost, roughly 19,000 emails per dollar and about one dollar per year at 50 daily. The cost unit is missing. If $0.026 is intended, the first ratio is about 1,923 per dollar and the annual cost about $9.49. The other figures imply roughly $0.0026 per batch. Treat the original as unresolved, not a budget. [H06; MISSING CONTEXT]

Troubleshoot with evidence

TEACHING CHOICE | Source-specific diagnostics from H12 D02 D20-D21

Symptom	Inspect	Next action
Claude ignores the skill	Installed target and loaded skill.	Invoke the documented skill explicitly; use D02's update and reload instructions.
Unauthorized API response	Provider, key presence and process environment.	Check configuration without logging the secret. Do not repeatedly retry a known invalid key.
Plausible but wrong labels	State, exclusions and actual raw output.	Review disagreements. Clarify criteria or add missing evidence before changing thresholds.
All records become one category	Option definitions and irrelevant context.	Add representative counterexamples and a valid unknown route; inspect input extraction.
A valid response triggers a bad action	Policy order, equality boundaries and missing fields.	Replay saved responses through the policy without live API calls.
Router works without a Jev key	Backend-ready log.	Distinguish built-in classification from a real Jev call; follow D20 configuration diagnostics.
No router lines in headless mode	Documented debug output.	Check the log path and plugin loading described by D20; no visible status row is not proof of failure.
Cost rises after model routing	Total tokens, cache behaviour and rework.	Disable main-model switching and compare against the saved baseline.
Hook output appears but Claude still runs	Hook return mode and request trace.	Context injection is not a short circuit. Inspect D21; test whether a model request occurred.
Browser repeats or misclicks	Observed element snapshot and actual final state.	Stop, refresh observations and validate the target; do not increase retries blindly.

Data boundaries to check before real use

Record which provider receives each payload: TypeSafe directly, a gateway, Claude, any speech or vision service, and any logging service. Installing a local client is not local inference. Minimize state and redact credentials before requests or logs. A Jev prompt-injection screen is not an access-control boundary.

H07's broad privacy claims are dated and unsupported as a complete account of current service terms. Consult TypeSafe's linked legal documents [D23] and the terms applicable to your account. Do not assume that one provider's retention arrangement covers a gateway or another model. This handbook does not certify suitability for confidential data.

Disable and recover. Keep a workflow bypass, restore the last known configuration, retain incomplete IDs and verify that the replacement path is functioning. For tools with side effects, reconcile completed actions before retrying.

Decide whether GPT 6 Astra adds value

EXTERNAL VERIFICATION D22 | INTERPRETATION and TEACHING CHOICE

Keep Claude as the default in this handbook. The supplied material supports a planner plus decision-model pattern, but it does not establish that GPT-6 Astra outperforms Claude on the proposed workflows. Adding Astra is justified by a measured improvement on a specific difficult task, not by the presence of a second provider.

OpenAI documents GPT-6 Astra for complex reasoning, coding and computer use, with function calling and structured outputs. That makes it a plausible planner or reviewer around Jev. These are documented capabilities, not evidence of a Jev-specific advantage over Claude. See the GPT-6 Astra model page [D22] for current API details, access requirements and pricing.

Possible role	Potential additional benefit	Evidence needed
Occasional planner	Decompose a difficult objective or revise a strategy after setbacks.	Better objective completion or fewer costly failures than the Claude baseline.
Independent reviewer	Challenge rubric assumptions or inspect disputed examples.	Useful disagreements resolved against human labels or executable evidence.
Builder for a specific subsystem	Implement or debug a component where Claude is struggling.	A working, tested result with acceptable total cost and rework.
Every-record processing	Usually adds another expensive step to a bounded task.	Clear quality improvement large enough to justify the added call.

Try it without redesigning the entire system

Keep the Jev adapter, state schema and policy fixed. Select a small set of hard cases that Claude handles poorly. Give Astra the same task, evidence and acceptance tests. Compare outcomes blind where possible. Add an optional planner interface only if the experiment shows a benefit; do not alternate models merely for variety.

Prompt to adapt

Review this Jev workflow as an independent planner. Use the same specification, labelled failures and acceptance tests supplied to Claude. Identify missing evidence, ambiguous criteria and broken policy assumptions. Propose a minimal revision with a test that can disprove its claimed benefit. Do not treat your judgment as ground truth or change production thresholds automatically.

Integration boundary

Use Astra through a supported environment or separate API adapter. The Claude Code router documented in H12 targets Claude tiers; it does not establish native switching from Claude to Astra. The official TypeSafe skill can guide other coding agents, but their installation, tools and credentials remain environment-specific. [D02, H12]

H04 alternates between “GPT-6 Astra” and “GPT-5.6 Astra” while describing a game demo. Preserve that identity uncertainty; the demonstration cannot serve as a model-specific comparison.

Source register for the supplied collection

PARAPHRASE | Stable local identifiers and line locators

Primary material: `jev_claude(2).txt`. Line numbers refer to the supplied plain-text copy. Most videos are pasted as a single long line. Descriptions below are neutral labels, not invented titles. Speaker names, original URLs and recording dates remain unverified unless stated.

Source and locator	Substantive contribution	Narrative and limits
H01 line 1 Launch and personal-app discussion	Classification examples and a cautious recommendation to evaluate before rollout.	Headline speed and cost claims; promotes a health app. Health examples are not validated deployment guidance. No verified metadata.
H02 line 2 Agentic dashboard demonstration	Inbox categories, model routing, asset choice, image descriptions and voice integration.	Promotes a community and a speech product. Demo timings and comparative claims lack controlled methods. Jev does not produce the graphics or voice.
H03 line 3 Correction and simulation discussion	Separates app building from decision selection; illustrates bounded choices in simulated systems.	Strong rhetorical framing. Simulations do not establish traffic-control or autonomous-driving suitability. No verified metadata.
H04 lines 4-353 Engineering workflow tutorial	Primitives, coding-agent setup, comment review, qualitative linters, skill routing and game planner/controller split.	Promotes courses and sandbox services. Some results are reported demos; browser swarms and broad review benefits are proposals. Inconsistent Astra naming.
H05 line 354 Business automation walkthrough	Comments, lead scoring, churn prioritization, community posts, call classification and reusable skills.	Promotes a course community. Numerical claims lack reproduced datasets; browser search is described too broadly as booking. No verified original URL.
H06 line 355 Detailed support workflow tutorial	Action-first question design, support triage, separate drafting screen, parallel questions and policy thresholds.	Promotes channel content. Missing cost unit; some probability/confidence and score-scale language needs correction. Claimed Desktop parity is explicitly untested.

Use H04-H06 as practical inspiration, not executable specifications. The linked primary documentation resolves interface details. The handbook's exercises deliberately replace dramatic demonstrations with reviewable, read-only pilots.

Source register continued

PARAPHRASE | Narrative separated from technical content

Source and locator	Substantive contribution	Narrative and limits
H07 line 356 Routing and batch tests	Model-routing demos, metadata search and multi-question comment analysis. Explicitly notes that agreement is not a gold-label evaluation.	Personal recommendations and unverified broad privacy statements. Local installation is confused with local processing. Generic Jev on/off/status commands belong to a custom setup.
H08 line 357 Custom dashboard architecture	Visual layer, files and memory, reusable skills, scheduling, voice and headless agent execution.	Mentions Chase AI Plus in a self-promotion segment; individual identity not independently established. The promoted templates and full implementation are not supplied.
H09 line 358 Prompt hook and read-only query demo	UserPromptSubmit interception, finite tools and arguments, formatted data-query answers.	Claims a short setup, rapid access and extremely low cost. Hook source and full setup prompt are missing; no trace proves universal Claude-call elimination.
H10 lines 359-412 Connecting Jev to Claude Code, and What That Actually Gets You	Separates skill installation from runtime use; includes API example, confidence routing and evaluation advice.	Supplied date 2026-09-27. Author and original page URL not recovered. Some confidence-as-correctness language overstates the documentation.
H11 lines 413-437 Router announcement	Introduces the jev-model-router mod and links its page.	Promotional summary. Not independent corroboration of H12; both describe the same integration.
H12 lines 438-554 Router documentation extract	Backend selection, routing switches, thresholds, fallback, logging, privacy, options and installation.	Matches the linked maintainer page D20. Early-access behaviour and APIs may change. Local fallback privacy wording needs narrower interpretation.

How contributions were selected

Repeated launches, promotional benchmarks, course offers and unsupported “zero error” language are excluded from the operating instructions. Detailed primary installation and cookbook material is linked. The handbook adds the missing connective work: selecting a route, defining a contract, validating a pilot, separating components and proving that an optimization helped.

No original video identity was inferred from a familiar voice or marketing phrase. Where a transcript points to a link “below” but supplies no usable URL, that material remains missing. The practical exercises are explicitly teaching designs, not reconstructions claimed to reproduce the creator's private setup.

Documentation map for setup and design

EXTERNAL VERIFICATION | Accessed 1 October 2026

Use these pages for details already documented well. The links are intentionally the authoritative continuation of this handbook, not an instruction to search for an unnamed tutorial.

D01 Jev with coding agents
What integration means and why Jev is not a chat-model replacement.

D02 Agent skill
Installation, manual setup, updates, explicit invocation and common skill issues.

D03 Official skill repository
Inspect the publisher's skill and reference files before installation.

D04 Quick start
Playground, first call and authentication walkthrough.

D05 Client SDKs
Choose the maintained Python or JavaScript guide and API reference.

D06 Introduction and primitives
The three output types and independent question evaluation.

D07 Confidence
Distribution-derived confidence and application thresholds.

D08 Speculative fan out
Ask several independent questions on shared state in one request.

D09 Composite scoring
Combine separate rubric dimensions with explicit code weights.

D10 Models
Current IDs, aliases, pricing, input constraints, rate limits and customization.

D11 Jev 1.13 jaggedness
Known limits involving numbers, indirection, context and adversarial content.

Read the applicable model-version limitations, not only the quick start. A passing request validates access and request shape; it says little about the quality of the question or the safety of the downstream action.

Documentation map for practical extensions

EXTERNAL VERIFICATION | Accessed 1 October 2026

D12 Skill suggestion
Complete vendor example and evaluation; scope is the published Hermes setup.

D13 Classifying RAG passages
Filtering evidence before a generative answer.

D14 Double checking citations
Check source support; preserve uncertainty and provenance.

D15 Guardrails for LLMs
Input and output screening recipe; not a security guarantee.

D16 Function calling
Map bounded intent and arguments to typed functions. Its trading example is not a deployment recommendation.

D17 Jev Ultrafast
Browser installation, indexed controls, helper model, inspector and stated limits.

D18 Score reference
Rubric structure, return fields and score interpretation.

D19 State
Prepare the evidence each question evaluates.

D20 Jev Model Router
Third-party mod instructions, configuration options and diagnostics.

D21 Claude Code hooks reference
Hook events, prompt processing and return semantics. Start at UserPromptSubmit for H09.

D22 GPT 6 Astra model documentation
Official capabilities, model ID, API features and pricing.

D23 TypeSafe legal document index
Links to current service and data-handling terms; review the applicable agreement.

D24 Full documentation index
Locate additional cookbooks and current SDK reference pages without guessing URLs.

Primary documentation establishes documented behaviour, not independent success rates. H10's article title is preserved in the source register, but no original page link is supplied because its URL was not verified.

Reusable knowledge model

TEACHING CHOICE | Canonical terms scoped to this handbook

ID and category	Canonical entity and definition	Recognition cue and sources
E01 Actor	TypeSafe AI - provider of the Jev service.	Provider, not the model. D01 D10
E02 Model/System	Jev - model evaluating typed questions against supplied state.	Bounded judgment. D01 D06
E03 Model/System	Claude - generative model used for writing, analysis and coding.	Write and reason. H04-H06
E04 Model/System	Claude Code - coding-agent environment hosting tools and integrations.	Execution environment. D02 D21
E05 Model/System	TypeSafe skill - instructions for designing and building integrations.	Teach the agent. D02 D03
E06 Input	State - evidence available for the current evaluation.	What it can see. D19
E07 Input	Question contract - instruction plus options, criteria or rubric.	What to judge. H06 D06
E08 Output	Typed answer - Noul value, Choice result or Score result.	Result, not action. D06 D18
E09 Concept	Confidence - distribution statistic returned for Choice and Score.	Concentration signal. D07
E10 Process	Policy gate - application logic mapping outputs to handling.	Code decides what happens. H06 D07
E11 Model/System	Approved executor - code or tool that performs an allowed action.	Actually does the work. H09 D16
E12 Evidence	Evaluation set - records with independently established reference outcomes.	Check against something. H07 H10
E13 Model/System	GPT-6 Astra - optional generative planner or reviewer.	Second model, measured role. D22
E14 Constraint	Missing evidence - required information absent from the state.	Unknown is not no. Teaching choice

Aliases. “Jeff”, “Jeb” and similar forms are normalized to Jev where context establishes the referent. “Cloud Code” is normalized to Claude Code in the relevant setup passages. “Null” and “bool” are transcript variants for the yes/no concept; the native TypeSafe primitive is Noul. Provider-specific boolean schemas remain distinct. “Agentic OS” denotes the surrounding custom application, not Jev. [H01-H12, D06]

Relationships and integrity audit

TEACHING CHOICE for the model | Evidence status preserved

Subject	Relationship	Object	Status and source
E02 Jev	CREATED_BY	E01 TypeSafe AI	Externally verified D01
E02 Jev	RECEIVES_INPUT	E06 State and E07 Question	Externally verified D06 D19
E02 Jev	PRODUCES	E08 Typed answer	Externally verified D06
E09 Confidence	PART_OF	Choice and Score answers	Externally verified D07
E10 Policy gate	USES	E08 Typed answer	Source-stated H06
E10 Policy gate	PRECEDES	E11 Approved executor	Interpreted from H09
E05 TypeSafe skill	ENABLES	Integration authoring in E04	Externally verified D02
E12 Evaluation set	QUALIFIED_BY	Domain and error costs	Teaching choice
E13 Astra	COMPARED_WITH	E03 Claude	Incomplete: no matched trial

Prerequisite map

Understand the action and its consequences before defining the question. Define the record and required evidence before selecting a primitive. Understand the answer fields before writing thresholds. Establish reference labels before measuring quality. Validate the complete pipeline before scheduling or adding a dashboard. Measure a stable baseline before adding a router or a second planner.

Integrity corrections that affect use

Output validity. Choosing only allowed labels does not establish truth. **Probability.** Low P(yes) is not low confidence. **Scores.** A rubric is not a calibrated event forecast. **Execution.** A returned action is not a completed or authorized action. **Training.** Revising prompts does not establish retraining. **Input.** Media descriptions are not direct image understanding. **Cost.** Free output does not mean free requests. [H02-H10; D06 D07 D10 D11]

Disagreements retained. Verification is useful only when narrowly scoped and grounded; sources disagree when they use that word for whole-answer certification. Model routing, effort routing and deterministic query handling are separate behaviours. Shared questions avoid repeated state but do not remove context-size limits or guarantee consistent answers across differently phrased questions. [H04 H07 H09 H12; D08 D11]

Still unresolved. Private demo source code, most video metadata, exact historical benchmark conditions, H06's cost unit, H09's complete hook semantics and any Astra-versus-Claude advantage on these workflows. Resolve these with original implementations and matched tests rather than promotional repetition.

Ready for practical use. Keep one inspectable workflow: known inputs, versioned questions, explicit policy, complete output accounting, evidence-linked review and a working fallback. Expand only when the measured result supports it.